Tutorial 06 para hacer en clase – Recuerde subir su enlace del repositorio antes de finalizar la clase.

Antes de iniciar:

- Recuerde terminar los tutoriales anteriores (ya que son acumulativos).
- Este tutorial muestra un ejemplo de creación de un proyecto "backend" con el framework "Nest.js".

A.Creación de un proyecto Nest.js

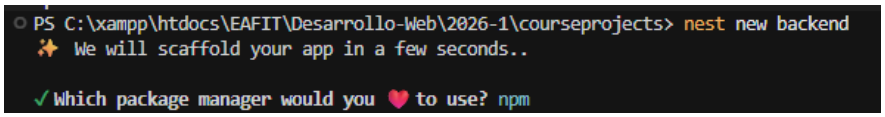
Crear un nuevo proyecto Nest.js:

- Abra la terminal, y en ubicado en la carpeta "courseprojects", ejecute los siguientes comandos:

Ejecutar en Terminal

```
npm i -g @nestjs/cli
nest new backend
```

- Seleccionan "npm" (presionan "enter").



Añadir prefijo "api"

- Ubicado en la carpeta raíz del proyecto ("backend"), modifique el archivo *src/main.ts* según lo que se resalta en negrilla en el siguiente código:

Modifique según la negrilla

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  app.setGlobalPrefix('api');

  await app.listen(process.env.PORT ?? 3000);
}
bootstrap();
```

Correr el proyecto Nest.js:

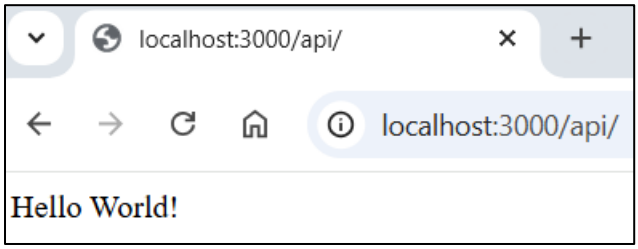
- Ubicado en la carpeta “courseprojects”, ejecute los siguientes comandos:

Ejecutar en Terminal

```
cd backend
npm run start
```

```
PS C:\xampp\htdocs\EAFIT\Desarrollo-Web\2026-1\courseprojects\backend> npm run start

> backend@0.0.1 start
> nest start

[Nest] 31288  - 03/13/2026, 11:57:29 AM     LOG [NestFactory] Starting Nest application...
[Nest] 31288  - 03/13/2026, 11:57:29 AM     LOG [InstanceLoader] AppModule dependencies initialized +9ms
[Nest] 31288  - 03/13/2026, 11:57:29 AM     LOG [RoutesResolver] AppController {/}: +3ms
[Nest] 31288  - 03/13/2026, 11:57:29 AM     LOG [RouterExplorer] Mapped {/, GET} route +7ms
[Nest] 31288  - 03/13/2026, 11:57:29 AM     LOG [NestApplication] Nest application successfully started +2ms
```

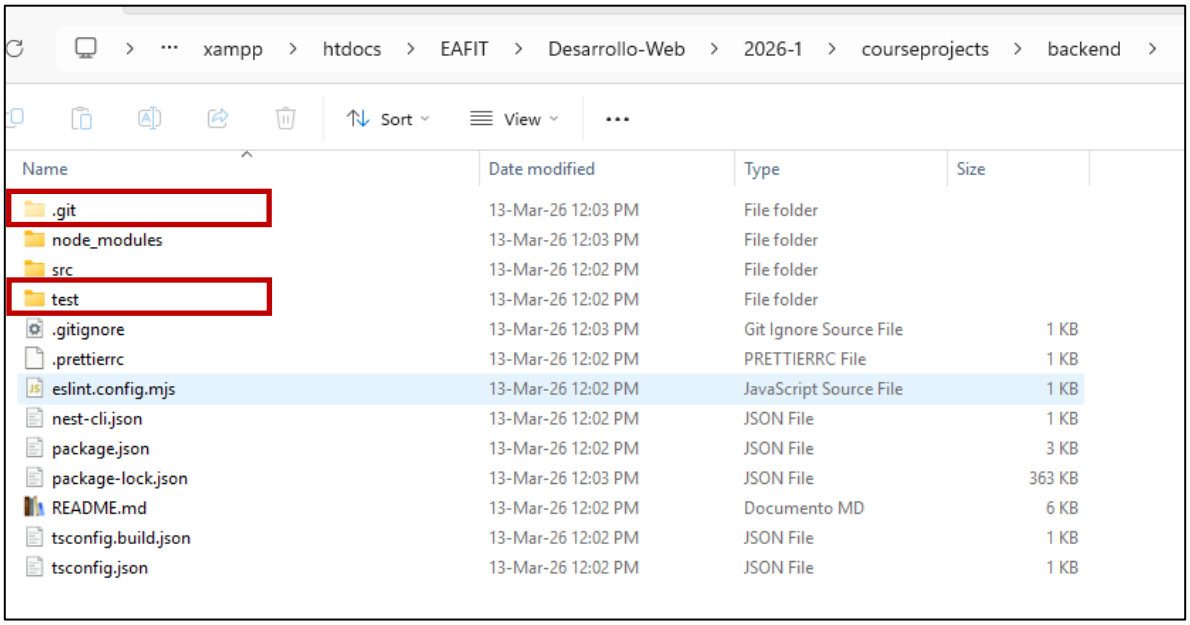
- Si todo sale bien, deberá poder acceder desde el navegador a la ruta http://localhost:3000/api/ y ver la página de inicio de la aplicación Nest.js (ver siguiente figura).



B. Configuración de Git

Configurar Git

- Ingresando por el explorador de archivos, vaya a la carpeta “backend”, muestre los archivos ocultos, y borre la carpeta “backend/.git”, además, borre la carpeta “backend/test/”:



- Ahora si podrá hacer “commits” sin problemas a su repositorio GitHub.

C. Configuración de Prettier y ESLint

Configurar Prettier y ESLint

- Ubicado en la carpeta raíz del proyecto ("backend"), modifique el archivo *package.json* según lo que se resalta en negrilla en el siguiente código:

Modifique según la negrilla

```
{
  ...
  "scripts": {
    "build": "nest build",
    "format": "prettier --write \"src/**/*.ts\" ~~\"test/**/*.ts\"~~",
    "start": "nest start",
    "start:dev": "nest start --watch",
    "start:debug": "nest start --debug --watch",
    "start:prod": "node dist/main",
    "lint": "eslint \"{src,apps,libs~~,test~~}/**/*.ts\" --fix",
...
```

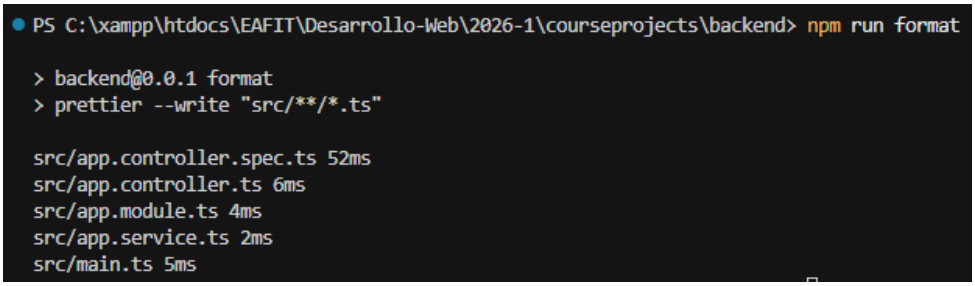
Ejecutar Prettier:

- Ubicado en la carpeta "frontend", ejecute el siguiente comando:

Ejecutar en Terminal

```
npm run format
```

- Si todo sale bien, deberá observar a "Prettier" modificando varios archivos de su proyecto (ver siguiente figura).



Ejecutar ESLint:

- Ubicado en la carpeta “backend”, ejecute el siguiente comando:

Ejecutar en Terminal

```
npm run lint
```

- Si todo sale bien, deberá observar a “ESLint” mostrando una alerta (ver siguiente figura).

```
PS C:\xampp\htdocs\EAFIT\Desarrollo-Web\2026-1\courseprojects\backend> npm run lint

> backend@0.0.1 lint
> eslint "{src,apps,libs}/**/*.ts" --fix


C:\xampp\htdocs\EAFIT\Desarrollo-Web\2026-1\courseprojects\backend\src\main.ts
  8:1  warning  Promises must be awaited, end with a call to .catch, end with a call to .then with a rejection handler or be explicitly marked as ignored with the `void`
 operator  @typescript-eslint/no-floating-promises

✖1 problem (0 errors, 1 warning)
```

Eliminar warning

- Ubicado en la carpeta raíz del proyecto (“backend”), modifique el archivo *src/main.ts* según lo que se resalta en negrilla en el siguiente código:

Modifique según la negrilla

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  app.setGlobalPrefix('api');

  await app.listen(process.env.PORT ?? 3000);
}
void bootstrap();
```

Ejecutar ESLint:

- Ubicado en la carpeta “backend”, ejecute el siguiente comando:

Ejecutar en Terminal

```
npm run lint
```

- Si todo sale bien, deberá observar a “ESLint” sin mostrar errores ni alertas (ver siguiente figura).

```
PS C:\xampp\htdocs\EAFIT\Desarrollo-Web\2026-1\courseprojects\backend> npm run lint

> backend@0.0.1 lint
> eslint "{src,apps,libs}/**/*.ts" --fix
```

D.Creación del módulo “home”

Borrar elementos que no vamos a utilizar por el momento:

- Ubicado en la carpeta “backend”, elimine los siguientes archivos y carpetas:
  - src/app.controller.spec.ts
  - src/app.controller.ts
  - src/app.service.ts

Modificar el app.module.ts

- Modifique el archivo *src/app.module.ts* reemplace todo el código, por el nuevo código que se presenta a continuación:

Reemplazar todo el código

```
import { Module } from '@nestjs/common';
import { HomeModule } from './home/home.module';

@Module({
  imports: [HomeModule],
})
export class AppModule {}
```

Crear módulo home

- Ubicado en la carpeta “src/”, cree la carpeta “home”, y en “src/home” cree el archivo *home.module.ts* con el siguiente contenido:

Añadir todo este código

```
import { Module } from '@nestjs/common';
import { HomeController } from './home.controller';

@Module({
  controllers: [HomeController],
})
export class HomeModule {}
```

Crear controlador home

- Ubicado en la carpeta “src/home” cree el archivo *home.controller.ts* con el siguiente contenido:

Añadir todo este código

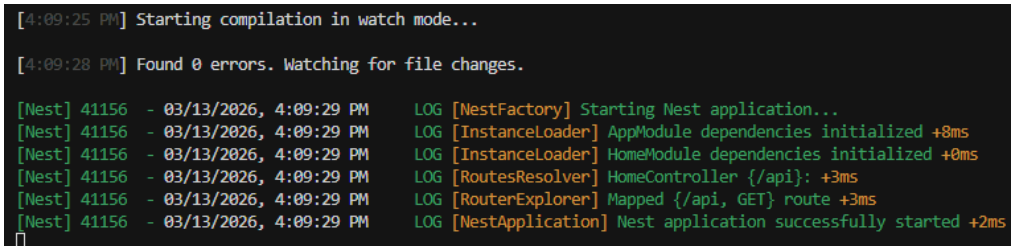
```
import { Controller, Get } from '@nestjs/common';

@Controller()
export class HomeController {
  @Get()
  index(): string {
    return 'API is running';
  }
}
```

Correr el proyecto:

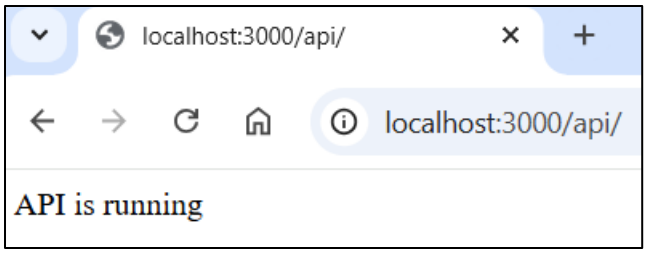
- Ubicado en la carpeta “backend”, ejecute el siguiente comando:

Ejecutar en Terminal

```
npm run start:dev
```



- **Nota:** el comando “start:dev” permite iniciar el proyecto en modo “escucha” de cambios. Si modifica un archivo, el servidor automáticamente se actualizará con los nuevos cambios.
- Si todo sale bien, deberá poder acceder desde el navegador a la ruta http://localhost:3000/api/ y ver la nueva página de inicio de la aplicación Nest.js (ver siguiente figura).



E. Creación del módulo “books”

Instalar TypeORM y SQLite:

- Abra la terminal, y en ubicado en la carpeta “backend”, ejecute el siguiente comando:

Ejecutar en Terminal

```
npm install @nestjs/typeorm typeorm sqlite3
```

Incluir TypeORM

- Ubicado en la carpeta raíz del proyecto (“backend”), modifique el archivo *src/app.module.ts* según lo que se resalta en negrilla en el siguiente código:

Modifique según la negrilla

```
import { Module } from '@nestjs/common';
import { HomeModule } from './home/home.module';
import { TypeOrmModule } from '@nestjs/typeorm';

@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'sqlite',
      database: 'database.sqlite',
      autoLoadEntities: true,
      synchronize: true,
    }),
    HomeModule
  ],
})
export class AppModule {}
```

Crear módulo books

- Ubicado en la carpeta “src/”, cree la carpeta “books”, y en “src/books” cree el archivo *books.module.ts* con el siguiente contenido:

Añadir todo este código

```
import { Module } from '@nestjs/common';
import { BooksController } from './books.controller';
import { BooksService } from './books.service';
import { TypeOrmModule } from '@nestjs/typeorm';
```

```
import { Book } from './entities/book.entity';

@Module({
  imports: [TypeOrmModule.forFeature([Book])],
  controllers: [BooksController],
  providers: [BooksService],
})
export class BooksModule {}
```

Crear controlador books

- Ubicado en la carpeta "src/books" cree el archivo *books.controller.ts* con el siguiente contenido:

Añadir todo este código

```
import { Controller, Get, Param, Post, Body } from '@nestjs/common';
import { BooksService } from './books.service';
import { Book } from './entities/book.entity';
import { CreateBookDto } from './dto/create-book.dto';

@Controller('books')
export class BooksController {
  constructor(private readonly booksService: BooksService) {}

  @Get()
  findAll(): Promise<Book[]> {
    return this.booksService.findAll();
  }

  @Get(':id')
  findOne(@Param('id') id: string): Promise<Book | null> {
    return this.booksService.findOne(Number(id));
  }

  @Post()
  create(@Body() createBookDto: CreateBookDto) : Promise<Book> {
    return this.booksService.create(createBookDto);
  }
}
```

Crear servicio books

- Ubicado en la carpeta “src/books” cree el archivo *books.service.ts* con el siguiente contenido:

Añadir todo este código

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Book } from './entities/book.entity';
import { CreateBookDto } from './dto/create-book.dto';

@Injectable()
export class BooksService {
  constructor(
    @InjectRepository(Book)
    private booksRepository: Repository<Book>,
  ) {}

  findAll(): Promise<Book[]> {
    return this.booksRepository.find();
  }

  findOne(id: number): Promise<Book | null> {
    return this.booksRepository.findOneBy({ id });
  }

  create(createBookDto: CreateBookDto): Promise<Book> {
    const book = this.booksRepository.create(createBookDto);
    return this.booksRepository.save(book);
  }
}
```

Crear entidad book

- Ubicado en la carpeta “src/books”, cree la carpeta “entities”. Luego, ubicado en “src/books/entities” cree el archivo *book.entity.ts* con el siguiente contenido:

Añadir todo este código

```
import { Entity, Column, PrimaryGeneratedColumn } from 'typeorm';

@Entity()
export class Book {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  title: string;

  @Column()
  category: string;

  @Column()
  price: number;

  @Column()
  stock: number;
}
```

Crear dto para la creación de libros

- Ubicado en la carpeta “src/books”, cree la carpeta “dto”. Luego, ubicado en “src/books/dto” cree el archivo *create-book.dto.ts* con el siguiente contenido:

Añadir todo este código

```
export class CreateBookDto {
  title: string;
  category: string;
  price: number;
  stock: number;
}
```

Incluir módulo books

- Ubicado en la carpeta raíz del proyecto ("backend"), modifique el archivo *src/app.module.ts* según lo que se resalta en negrilla en el siguiente código:

Modifique según la negrilla

```
import { Module } from '@nestjs/common';
import { HomeModule } from './home/home.module';
import { TypeOrmModule } from '@nestjs/typeorm';
import { BooksModule } from './books/books.module';

@Module({
  imports: [
    ...,
    HomeModule,
    BooksModule,
  ],
})
export class AppModule {}
```

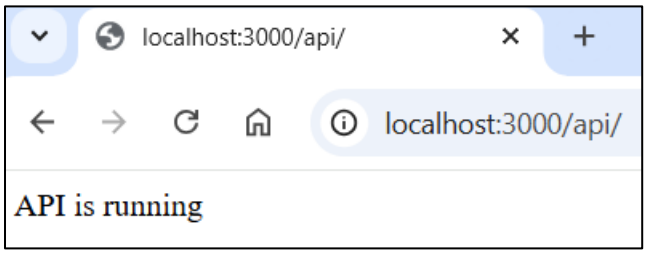
Correr el proyecto:

- Ubicado en la carpeta "backend", ejecute el siguiente comando:

Ejecutar en Terminal

```
npm run start:dev
```

- Si todo sale bien, deberá poder acceder desde el navegador a la ruta http://localhost:3000/api/ y ver la página de inicio de la aplicación Nest.js funcionando correctamente (ver siguiente figura).



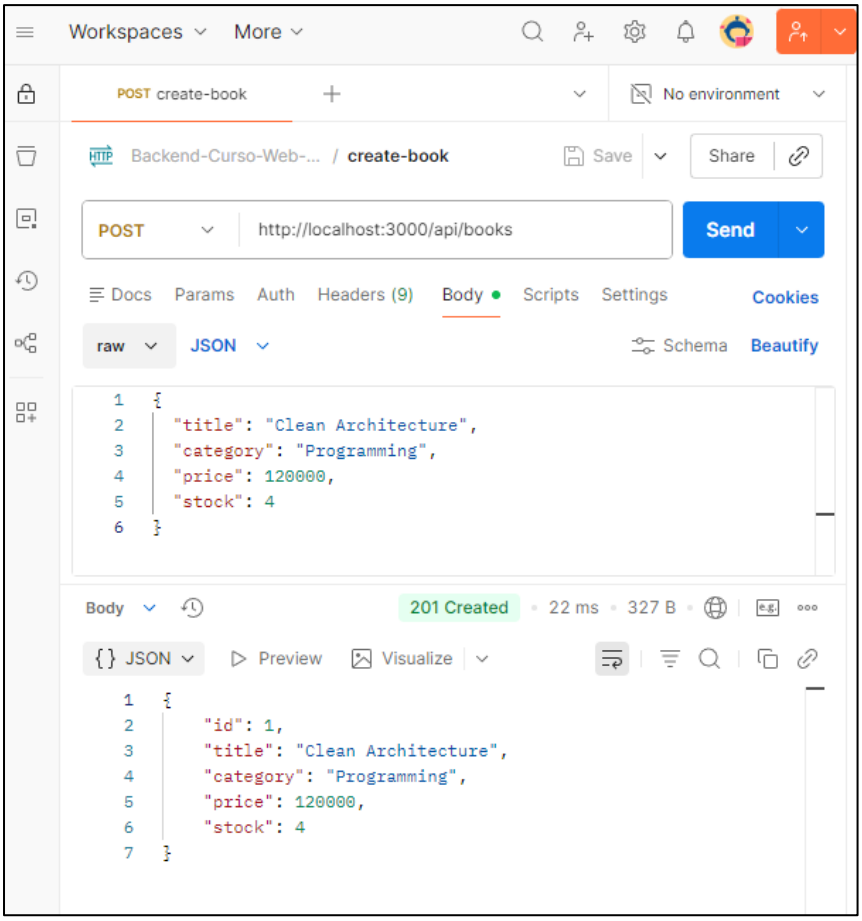
F. Testeando el backend

Herramientas clientes de API

- Descargue una herramienta ya sea “Postman” o “Insomnia”. Estas herramientas son programas que permiten enviar peticiones HTTP a un servidor para probar o desarrollar APIs.
- Una vez descargada la herramienta de su preferencia apréndala a utilizar buscando videos en YouTube o utilizando ChatGPT. Y luego conéctese con su proyecto “backend” para crear y listar libros.
- Yo les mostraré, a continuación, pantallazos de cómo he utilizado “Postman” para probar mi “backend”. Usted lo podría utilizar de base para sus pruebas.

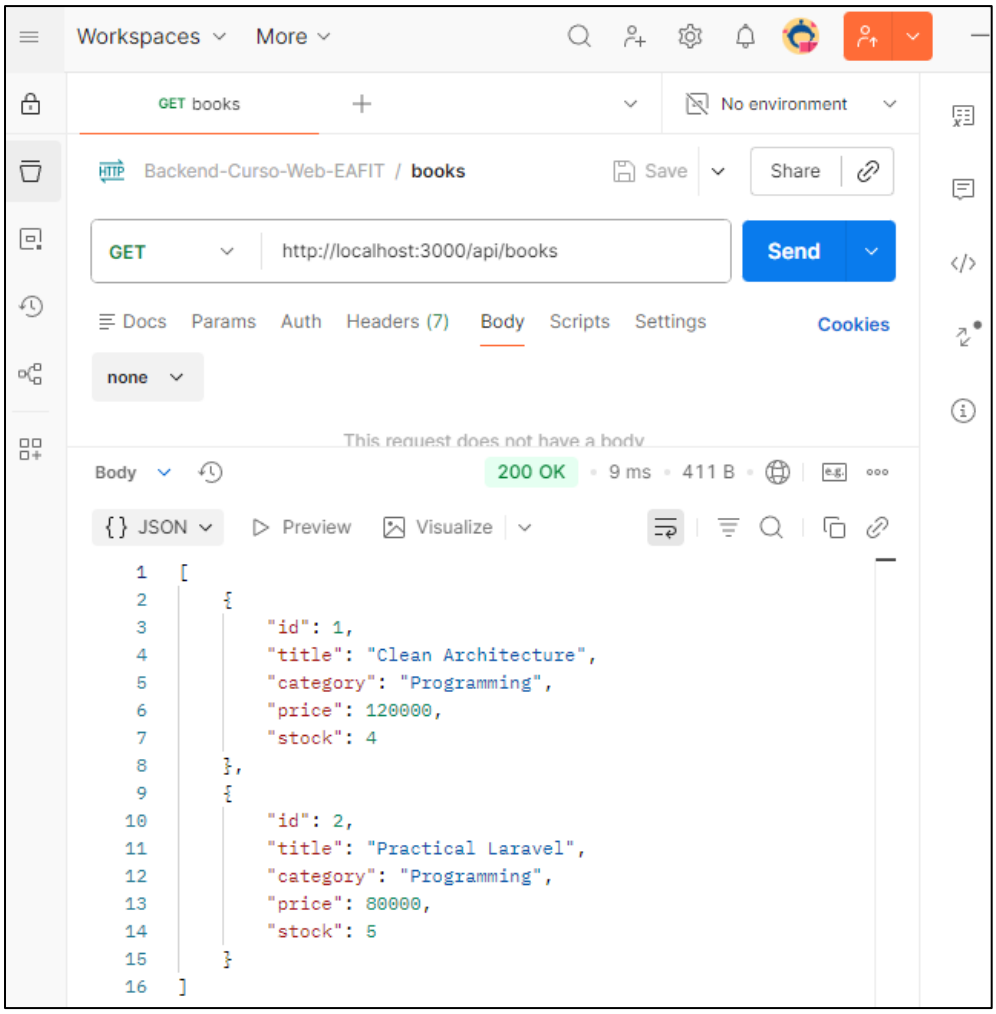
Crear libros

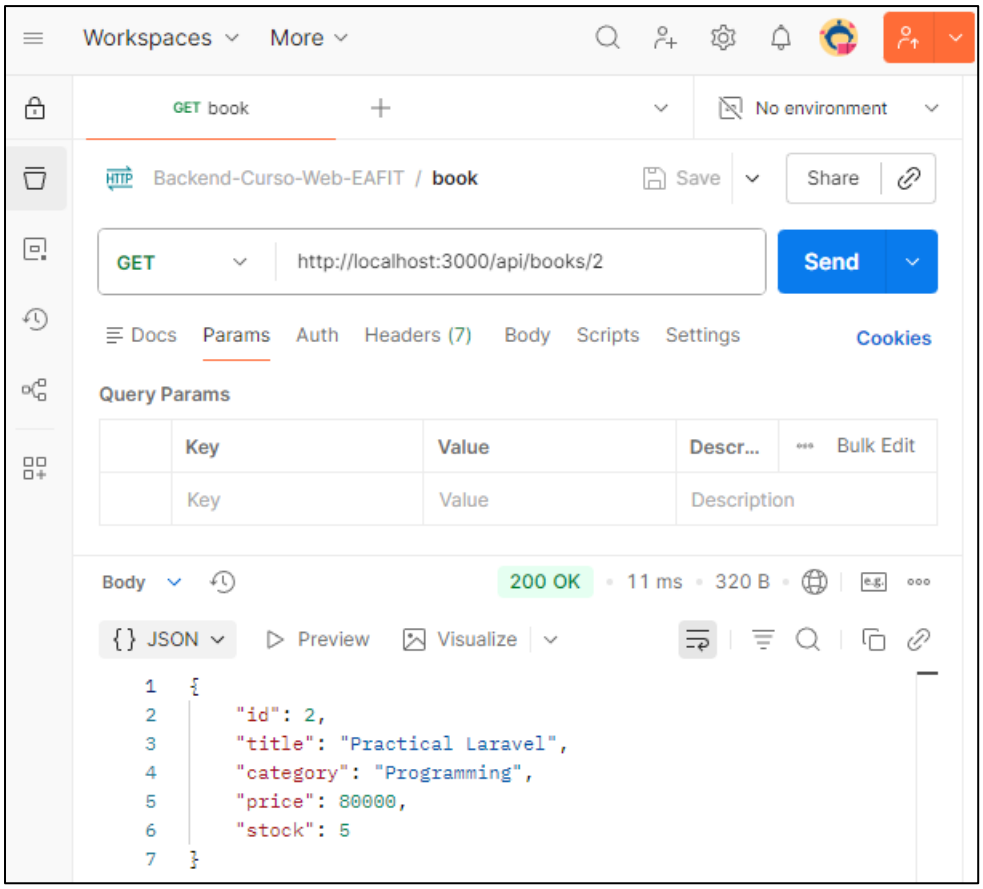
- Utilizando su herramienta cree un par de libros haciendo peticiones “post” a http://localhost:3000/api/books (ver siguiente ejemplo):



Ver libros

- Utilizando su herramienta liste los libros haciendo una petición “get” a http://localhost:3000/api/books (ver siguiente ejemplo):



Ver un libro

- Utilizando su herramienta liste un libro haciendo una petición "get" a http://localhost:3000/api/books/2 (ver siguiente ejemplo):